

INDEX

NAME K.D.D. Sai AbhiramSUBJECT Deep learning lab work

STD. _____ DIV. _____ ROLL NO. _____ SCHOOL _____

Sr. No.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1.		Analyze and exploring the deep learning platforms	24-09-23	He
2.		Implement a classifier using open-source data set	7/8/25	He 11/11/25
3.		study of the classifiers with respect to statistical parameters	7/8/25	He 11/11/25
4.		Build a simple feed forward neural network to recognize hand written characters	14/8/25	He 11/14/25
5.		Study of activation function and its role	28-8-23	He 11/9/23
6.		Implement gradient descent & back propagation in deep neural network.	09-9-23	He 11/9/23
7.		Build a CNN model to classify cats & dogs images	16-7-25	He 11/23/25
8.		Experiment using LSTM		He 11/17/25
9.		Build a RNN		He 11/17/25
10.		perform Compression on MNIST data set using Autoencoder		He 11/17/25
11.		to implement experiment using variational Autoencoder		He 11/17/25
12.		Implement DCGAN to generate colour images	22/10	He 11/21/23
13.		understanding the architecture of Pre-trained model	22/10	He 11/21/23
14.		Pre-trained CNN model as feature extractor using TM		He 11/21/23
15.		Implement a YOLO model to detect objects		He 11/21/23

samples ~~He~~

19. Implement a deep Convolutional GAN to generate complex colour images [DCGAN]

Aim:- to implement a deep convolutional Generative adversarial network to generate complex colour images.

Objective:

- ① To understand and apply the architecture of DCGAN using convolution and transposed convolutional layers.
- ② open data set as a source of complex real world images
- ③ to generate to produce synthetic images from random noise
- ④ to train the discriminator to differentiate between real and generated images.
- ⑤ to evaluate the GAN model through visualization.

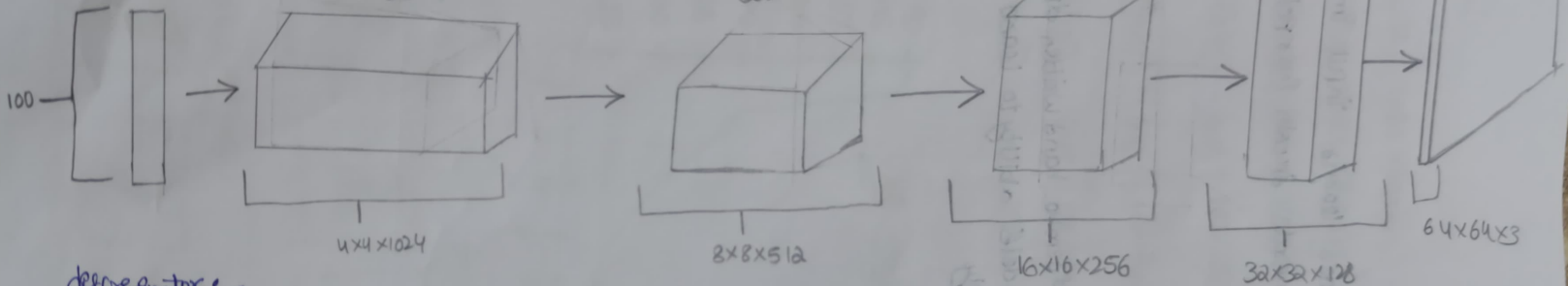
Pseudo Code

- ① Initialize dimensions, learning rate, batch size, epochs
- ② load data set and normalize images
- ③ define Generator : relu activation and batch normalization
- ④ define discriminator : use conv2d layers with leaky relu and batch normalization
- ⑤ Initialize optimizer and loss function
- ⑥ for epoch each
 - Ⓐ train discriminator
 - Ⓑ train generator.
- ⑦ save losses and visualize generated images periodically

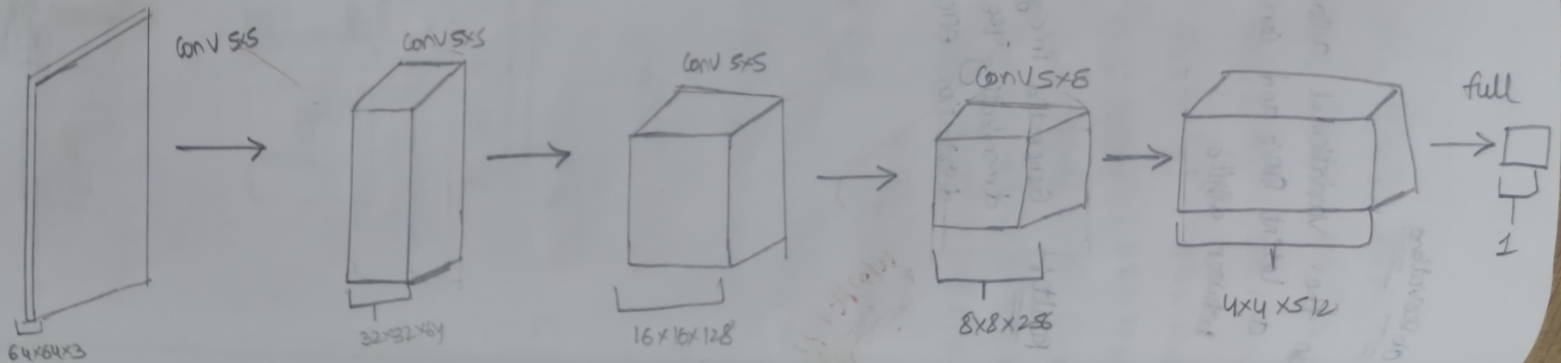
Deep Convolutional GAN

Generator :-

Project and reshape



Discriminator :-





()



LAB 12: DCGAN with Loss & Accuracy Visualization

```
import tensorflow as tf
from tensorflow.keras import layers
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
from IPython import display
```

1. Load CIFAR-10 open dataset

```
(ds_train, ds_info), _ = tfds.load(
    'cifar10',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
)
```

```
def preprocess(image, label):
    image = tf.cast(image, tf.float32)
    image = (image - 127.5) / 127.5
    return image
```

```
BATCH_SIZE = 64
BUFFER_SIZE = 10000
```

```
train_dataset = ds_train.map(preprocess).shuffle(BUFFER_SIZE).batch(BATCH_SIZE).take(100)
```

5. Loss functions

```
def generator_loss(fake_output):  
    return cross_entropy(tf.ones_like(fake_output), fake_output)  
  
def discriminator_loss(real_output, fake_output):  
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)  
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)  
    return real_loss + fake_loss
```

6. Training step with accuracy

noise_dim = 100

@tf.function

```
def train_step(images):  
    noise = tf.random.normal([BATCH_SIZE, noise_dim])  
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:  
        generated_images = generator(noise, training=True)  
        real_output = discriminator(images, training=True)  
        fake_output = discriminator(generated_images, training=True)  
        gen_loss = generator_loss(fake_output)  
        disc_loss = discriminator_loss(real_output, fake_output)  
  
    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)  
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)  
  
    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))  
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))  
  
    # Discriminator accuracy  
    real_acc = tf.reduce_mean(tf.cast(real_output > 0, tf.float32))  
    fake_acc = tf.reduce_mean(tf.cast(fake_output < 0, tf.float32))  
    disc_acc = (real_acc + fake_acc) / 2.0  
    return gen_loss, disc_loss, disc_acc
```

2. Build Generator

```
def build_generator():  
    model = tf.keras.Sequential([  
        layers.Input(shape=(100,)),  
        layers.Dense(8*8*256, use_bias=False),  
        layers.BatchNormalization(),  
        layers.LeakyReLU(),  
        layers.Reshape((8,8,256)),  
        layers.Conv2DTranspose(128,(5,5), strides=(2,2), padding='same', use_bias=False),  
        layers.BatchNormalization(),  
        layers.LeakyReLU(),  
        layers.Conv2DTranspose(64,(5,5), strides=(2,2), padding='same', use_bias=False),  
        layers.BatchNormalization(),  
        layers.LeakyReLU(),  
        layers.Conv2DTranspose(3,(5,5), strides=(1,1), padding='same', use_bias=False, activation='tanh')  
    ])  
    return model
```

3. Build Discriminator

```
def build_discriminator():  
    model = tf.keras.Sequential([  
        layers.Input(shape=(32,32,3)),  
        layers.Conv2D(64,(5,5), strides=(2,2), padding='same'),  
        layers.LeakyReLU(),  
        layers.Dropout(0.3),  
        layers.Conv2D(128,(5,5), strides=(2,2), padding='same'),  
        layers.LeakyReLU(),  
        layers.Dropout(0.3),  
        layers.Flatten(),  
        layers.Dense(1)  
    ])  
    return model
```

4. Initialize models & optimizers

```
generator = build_generator()  
discriminator = build_discriminator()  
  
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)  
generator_optimizer = tf.keras.optimizers.Adam(1e-4)  
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

```
# 9. Plot Loss & Accuracy
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(range(1,EPOCHS+1), gen_losses, label='Generator Loss')
```

```
plt.plot(range(1,EPOCHS+1), disc_losses, label='Discriminator Loss')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.title('DCGAN Loss Curves')
```

```
plt.legend()
```

```
plt.subplot(1,2,2)
```

```
plt.plot(range(1,EPOCHS+1), disc_accuracies, label='Discriminator Accuracy', color='green')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.title('Discriminator Accuracy (%)')
```

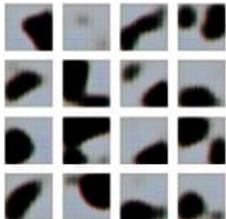
```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```



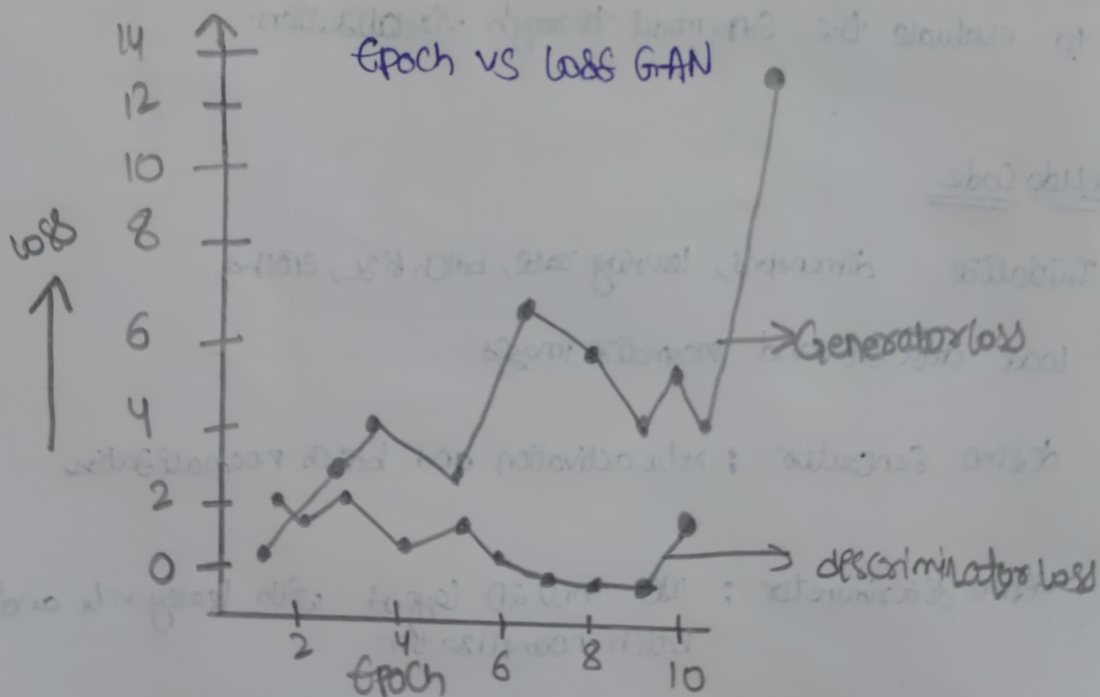
Epoch 10



Epoch 10 completed

output

Epoch [1/10]	D loss : 1.0992	; G loss : 0.5678
Epoch [2/10]	D loss : 0.5060	; G loss : 2.7113
Epoch [3/10]	D loss : 1.2431	; G loss : 3.9824
Epoch [4/10]	D loss : 0.4229	; G loss : 2.264
Epoch [5/10]	D loss : 1.1486	; G loss : 5.1007
Epoch [6/10]	D loss : 1.0081	; G loss : 6.7625
Epoch [7/10]	D loss : 0.0252	; G loss : 4.578
Epoch [8/10]	D loss : 0.0043	; G loss : 6.3371
Epoch [9/10]	D loss : 0.1380	; G loss : 4.7357
Epoch [10/10]	D loss : 2.3294	; G loss : 14.0363



Observations

- ① Loss curves stabilize gradually $\rightarrow$ showing adversarial balance
- ② Generated images evolve from random noise $\rightarrow$ structured forms $\rightarrow$ realistic sample

Result

Successfully Implemented the DCGAN effectively.

~~eff~~